

Задача 6. Ускорение уже реализованного кода

Есть модуль, который конвертирует html в JSON-документ особого вида. В объекте есть два основных поля – `objects` и `styles`.

`objects` – массив объектов, полученных из блоков `<p>` или `<table>` исходного документа.

`styles` – объект стилей, аналогичный реализованному в задаче 2 (парсинг html).

Задача: ускорить и оптимизировать алгоритм, в том числе убрав из него использование библиотеки для работы с html «cheerio» (на пртм можно посмотреть, что это).

Особенности:

- 1) При создании объекта из блока `<p>` (см. `get-paragraph.js`) заполняются следующие поля:
 - `type<string>` - принимается равным «P»;
 - `source<string>` - исходная html-строка, из которой получен данный объект;
 - `pid<string>` - строка с номером `pid`'а, заполняется аналогично задаче 2;
 - `spacesBefore<number>` - в оригинале задумывалось что здесь будет храниться информация о том, какой отступ текста от левой стороны страницы, но в итоге заполняется только в том случае если отступ равен 5, (пока?) делаем аналогично;
 - `text<string>` – чистый текст без тегов (кроме картинок, текст с картинками обрабатывается отдельно, см. `get-paragraph:62-70`);
 - `format<string>` - формат текста («`formattext`»/ «`unformattext`»/«`headertext`»/«`»`») – заполняется аналогично реализации в `text-format-utils.js`;
 - `align<string|undefined>` - выравнивание, заполняем если есть;
 - `isBold<boolean>` – показывает, является ли текст жирным, см. `get-paragraph:38`, `text-format-utils.js` (здесь внимательно смотреть реализацию);
 - `isItalic<boolean>` - аналогично `isBold`, но с курсивом;
 - `isBoldBegin<boolean>` - показывает, начинается ли текст с жирного текста
 - `docType<string>` - всегда равен «`kdoc`».
- 2) При создании объектов из `<table>`, в основном заполняем:
 - `type<string>` - принимается равным «TABLE»;
 - `cells<array>` - массив `p`-объектов изнутри таблицы;
 - `firstPid`, `lastPid` - аналогично заданию 2;
 - `docType` = «`kdoc`».Но таблицы не всегда реально являются таблицами... (см. следующие пункты).
- 3) При создании объектов из `<table>` (см. `get-tables.js`) могут происходить разные стрёмные вещи, а именно:
 - это может быть не таблица, а рамка/двойная рамка, такие случае нужно обрабатывать аналогично обработке в коде
 - это может быть не таблица, а просто элемент разметки, то есть все границы отсутствуют, тогда разбиваем всё на ячейки (то есть просто блоки `<p>`, которые лежат внутри) или строки таблицы (тип «`ROW`», о нём чуть позже);
 - это может быть перечень терминов, которые также выровнены с использованием невидимой таблицы (обычно номер-определение-иностранное название), тогда

- делаем из них объекты, выкидывая иностранное название;
- и тд, насчёт таблиц лучше поговорить вживую если по коду будет непонятно;
- 4) Тип «ROW» при заполнении имеет:
type = «ROW»
source<string> - исходная html-строка (полученная из <tr></tr>);
text – чистый текст всех ячеек, «склеенный» через пробелы, с картинками аналогично <p>
cells, firstPid, lastPid – аналогично table.
- 5) Для автоматического тестирования используем jest как в первой задаче. К задаче будут прикреплены 40 файлов, которые должны при обработке конвертироваться в JSON-файлы, идентичные приложенным.
- 6) Естественно, не пишем всё в одном файле, а разбиваем на модули, нужна хорошая архитектура.
- 7) Пишем JSDoc-комментарии к каждому классу/функции.
- 8) По возможности реализовать классы для p-, table- и row-объектов.
- 9) По оформлению – если модуль является классом, то называем его с большой буквы, используя camel case, если нет, то kebab case, сам код делаем по стандартам prettier.
- 10) Важно следить за безопасностью регулярных выражений (чтобы они не уходили в бесконечный или крайне долгий цикл), а также за тем чтобы в случае косяков в самих документах скрипт не падал, а обрабатывал хотя бы какую-то часть (последнее возможно не понадобится, эту часть обсудим после реализации всего остального).
- 11) Для создания JSON было использовалось форматирование JSON.stringify(object, null, 2).